Unit 5: Array

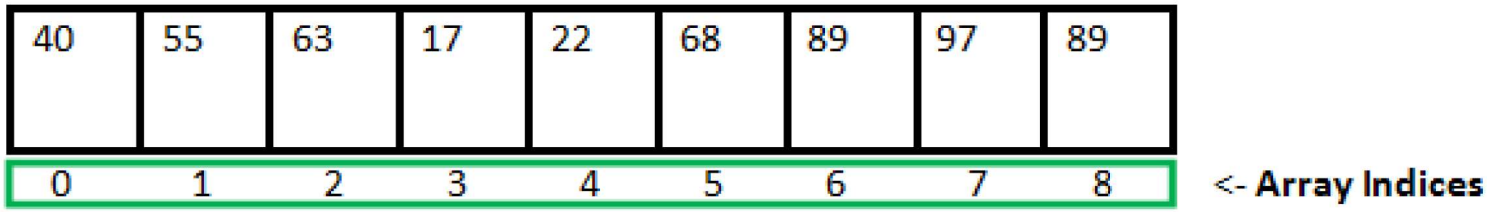
Compiled by: Er. Krishna Khadka

Array

- An array in C or be it in any programming language is a collection of similar data items stored at contiguous memory locations and elements can be accessed randomly using indices of an array.
- They can be used to store collection of primitive data types such as int, float, double, char, etc of any particular type.
- To add to it, an array in C can store derived data types such as the structures, pointers etc.

Array

- Given below is the picturesque representation of an array.



Array Length = 9
First Index = 0
Last Index = 8

Why do we need arrays?

- We can use normal variables (v1, v2, v3, ..) when we have a small number of objects, but if we want to store a large number of instances, it becomes difficult to manage them with normal variables.
- The idea of an array is to represent many instances in one variable.

One Dimensional Array declaration in C:

- Linear list of fixed number of data items of same type.
- All these data items are accessed using same name using single subscript.

Declaration of array:

data_type arrayname[size];

Eg. int num [10];
float marks[5];
char name [30]; //string

Syntax: data_type name of the array [no. of elements];

For example: an array of integers can be declared as follows:

```
int arr[5];
```

Compiler will allocate a contiguous block of memory of size = `5*sizeof(int)`

Array declaration:

// Array declaration by specifying size

int arr1[10];

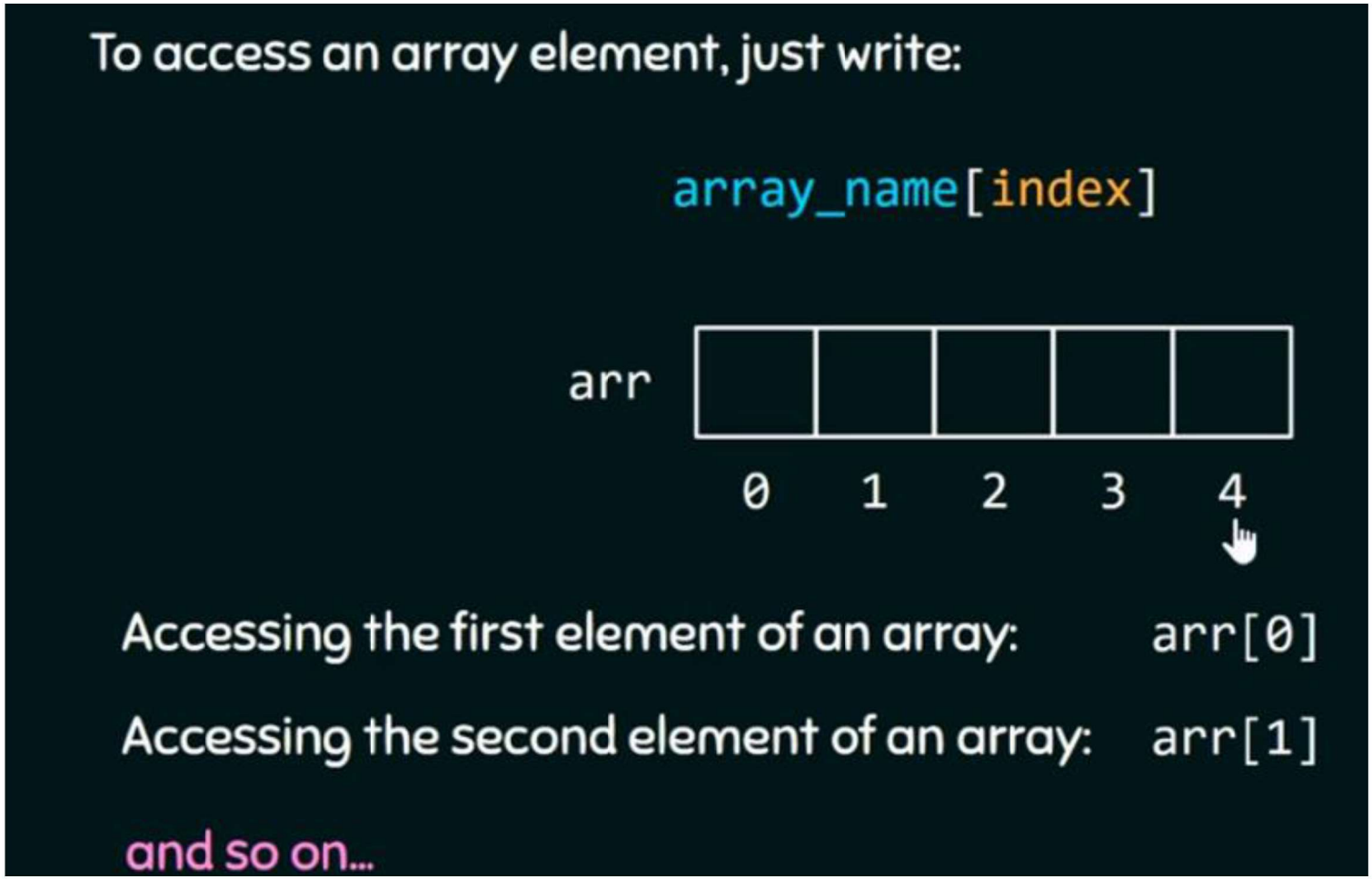
// Array declaration by initializing elements

int arr[] = { 10, 20, 30, 40 }

// Compiler creates an array of size 4.

// above is same as **"int arr[4] = {10, 20, 30, 40}"**

Accessing elements form 1D array



One Dimensional Array: contd..

Initialization:

data_type arrayname[size]= { list of variables seperaed by ,};

Eg. int num [5]= {2, 6, 3, 8, 12};

0 is the base index.

while access array elements,

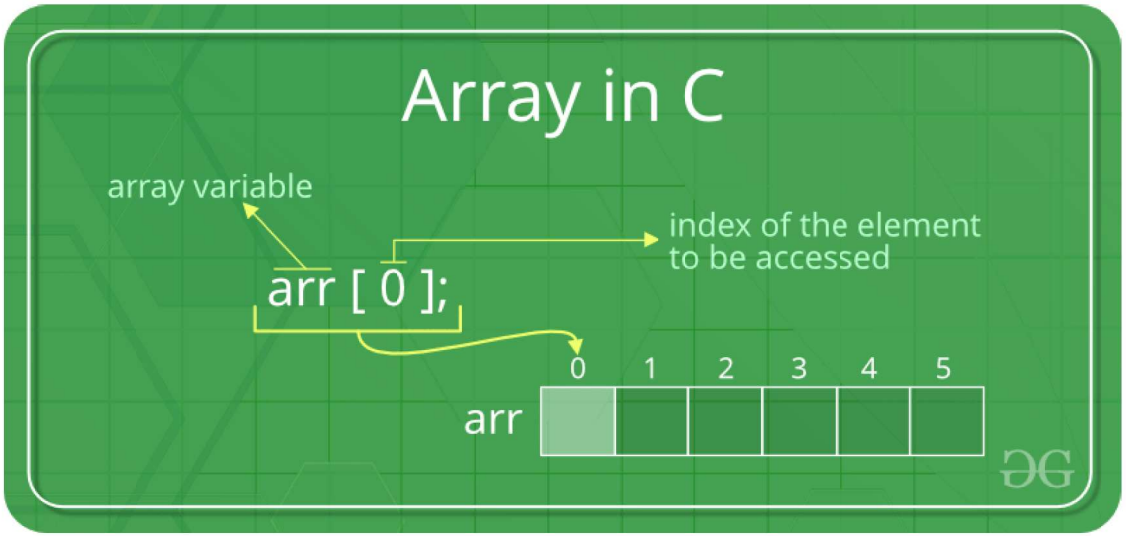
num[0]=? 2
num[3]=? 8
num[2]=? 3
num[4]=? 12

Compile time initialization.

Can also be initialized at run time also.

Facts about Array in C:

- **Accessing Array Elements:**
  Array elements are accessed by using an integer index.
  Array index starts with 0 and goes till size of array minus 1.
- Name of the array is also a pointer to the first element of array.



Advantages of an Array in C:

1. Random access of elements using array index.
2. Use of less line of code as it creates a single array of multiple elements.
3. Easy access to all the elements.
4. Traversal through the array becomes easy using a single loop.
5. Sorting becomes easy as it can be accomplished by writing less line of code.

Disadvantages of an Array in C:

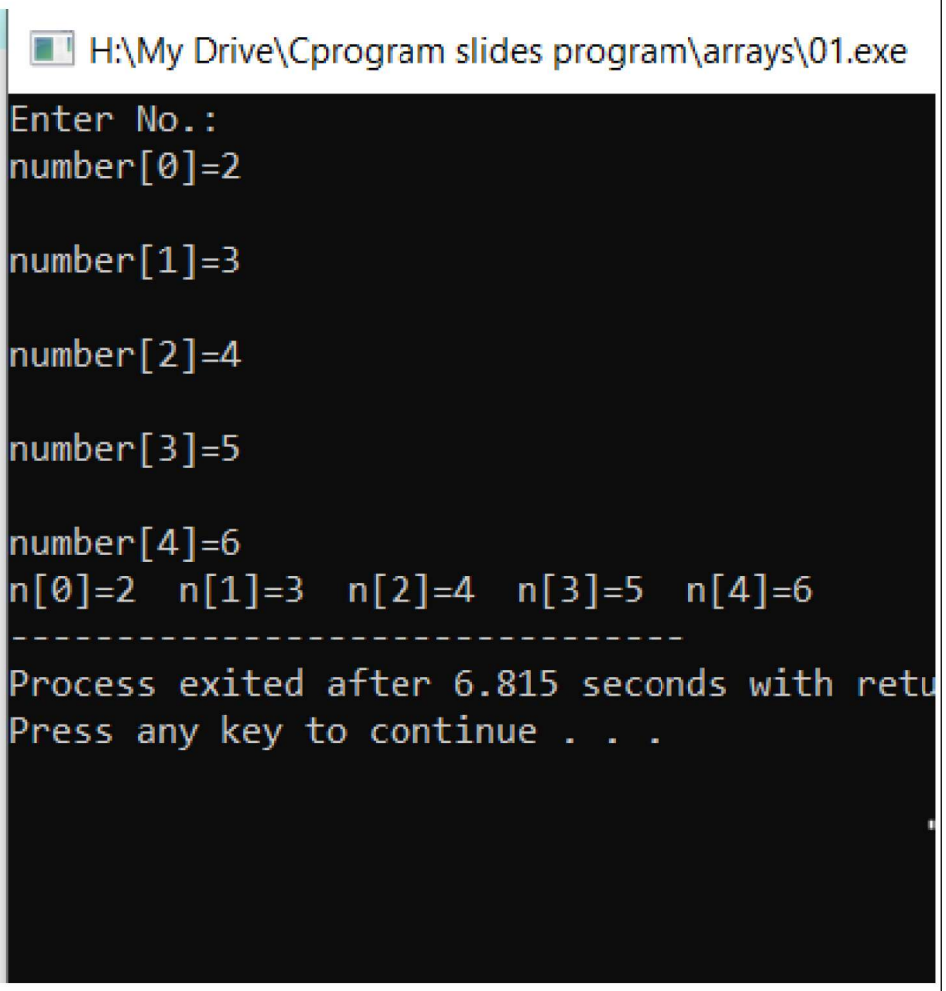
1. Allows a fixed number of elements to be entered which is decided at the time of declaration. Unlike a linked list, an array in C is not dynamic.
2. Insertion and deletion of elements can be costly since the elements are needed to be managed in accordance with the new memory allocation.

```c
//Write a program to read 5 numbers and display them.
#include<stdio.h>
#include<conio.h>
void main()
{
int num[5], i;
//clrscr();
printf("\n Input 20 data");
for(i=0;i<5;i++)
{
printf("\n num[%d]:- ", i);
scanf("%d", &num[i]);
}
printf("the numbers you have entered:\n");
for(i=0;i<5;i++)
{
printf("%5d", num[i]);
}
getch();
}
```

```
H:\My Drive\Cprogram slides program\arrays\1.exe

Input 20 data
num[0]:- 3

num[1]:- 2

num[2]:- 4

num[3]:- 6

num[4]:- 5
the numbers you have entered:
    3    2    4    6    5
```

Program to Display 5 numbers using array

```c
#include<stdio.h>
int main()
{
    int i , a[5];
    printf("Enter No.:");
    for(i=0;i<5;i++)
    {
        printf("\nnumber[%d]=",i);
        scanf("%d",&a[i]);
    }
    for(i=0;i<5;i++)
    {
        printf("n[%d]=%d\t",i,a[i]);
    }
    return 0;
}
```



Program to find Sum of 5 number using array

```c
#include<stdio.h>
int main()
{
    int i , a[5],sum=0;
    printf("Enter No.:\n");
    for(i=0;i<5;i++)
    {
        printf("n[%d]=",i);
        scanf("%d",&a[i]);
    }
    for(i=0;i<5;i++)
    {
        sum=sum+a[i];
    }
    printf("Sum=%d",sum);
    return 0;
}
```

```
H:\My Drive\Cprogram slides program\arrays\2.exe
Enter No.:
n[0]=2
n[1]=4
n[2]=3
n[3]=5
n[4]=1
Sum=15
--------------------------------
Process exited after 5.105 seconds with return value 0
Press any key to continue . . .
```

QN 3: Program to find the smallest and the largest element in the array

```c
#include<stdio.h>
int main()
{
    int i,n,a[100],small,large;
    printf("Enter value of n:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("n[%d]:",i);
        scanf("%d",&a[i]);
    }
    small=a[0];
    large=a[0];

    for(i=0;i<n;i++)
    {
        if(small>a[i])
            small=a[i];
        if(large<a[i])
            large=a[i];
    }
    printf("Small=%d\nLarge=%d",small,large);
    return 0;
}

```

```
H:\My Drive\Cprogram slides program\arrays\3.exe
Enter value of n:5
n[0]:2
n[1]:4
n[2]:3
n[3]:5
n[4]:6
Small=2
Large=6
--------------------------------
Process exited after 12.72 seconds with re
Press any key to continue . . .
```

QN 4: Program to sort elements using bubble sort

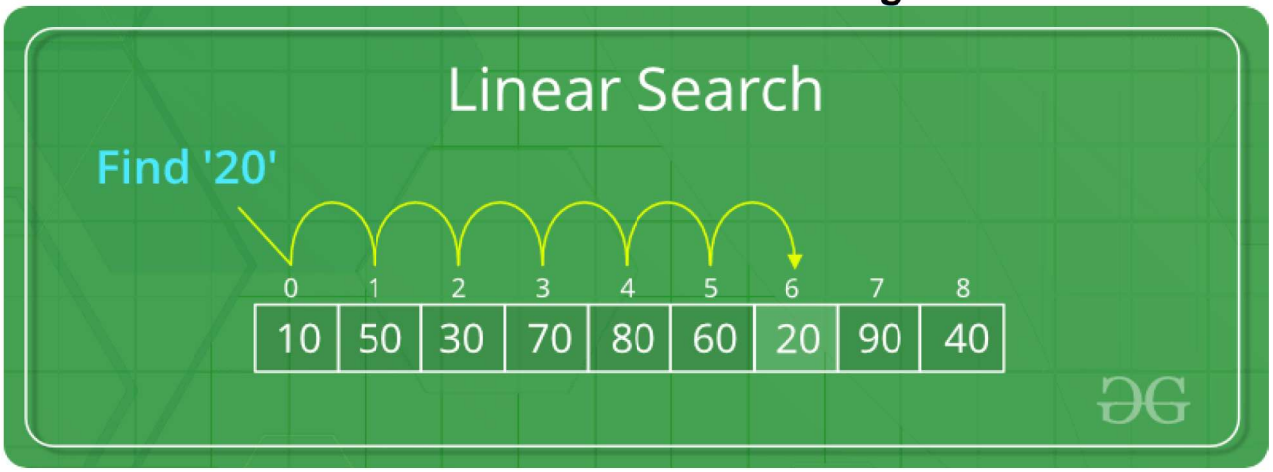
```
#include<stdio.h>
int main()
{
   int i,n,a[100],small,large,j,temp;
   printf("Enter value of n:");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("n[%d]:",i);
      scanf("%d",&a[i]);//taking input
   }
   for(i=0;i<n-1;i++)
   {
      for(j=0;j<n-i-1;j++)
      {
         if(a[j]>a[j+1])//sorting logic
          {
              temp=a[j];
              a[j]=a[j+1];
              a[j+1]=temp;
          }
        }
    }

 printf("Ascending:\n");
   for(i=0;i<n;i++)
      printf("%d\t",a[i]);
   printf("\nDescending:\n");
   for(i=n-1;i>=0;i--)
      printf("%d\t",a[i]);
   return 0;
}
```

```
Enter value of n:6
n[0]:1
n[1]:3
n[2]:2
n[3]:5
n[4]:4
n[5]:6
Ascending:
1       2       3       4       5       6
Descending:
6       5       4       3       2       1
```

Searching in an Array

- Searching Algorithms are designed to check for an element or retrieve an element from any data structure where it is stored.
- **Sequential Search**: In this, the list or array is traversed sequentially and every element is checked.

Linear Search to find the element "20" in a given list of numbers



QN 6: Program to search using array

```c
#include<stdio.h>
int main()
{
    int arr[]={1,3,5,7,20,12,23,78,98,67,56,45,19,65,9},key,i,flag=0;
    printf("\nENTER A NUMBER: ");
    scanf("%d",&key);
    for(i=0;i<10;i++)
    {
        if(key==arr[i])
        flag=1;
    }
    if(flag==1)
        printf("\nTHE NUMBER %d EXISTS IN THE ARRAY",key);
    else
        printf("\nTHE NUMBER %d DOES NOT EXIST IN THE ARRAY",key);
    return 0;
}
```

```
H:\My Drive\Cprogram slides program\Clabsolutions\lab3soln\6.exe

ENTER A NUMBER: 5

THE NUMBER 5 EXISTS IN THE ARRAY
--------------------------------
Process exited after 3.207 seconds with return value 0
Press any key to continue . . .
```

Multi-dimensional Arrays in C

- C programming language allows multidimensional arrays. Here is the general form of a multidimensional array declaration –

  type name[size1][size2]...[sizeN];

- For example, the following declaration creates a three dimensional integer array –

  int threedim[5][10][4];

Two-dimensional Arrays

- The simplest form of multidimensional array is the two-dimensional array.
- A two-dimensional array is, in essence, a list of one-dimensional arrays. To declare a two-dimensional integer array of size [x][y], you would write something as follows –

  type arrayName [x][y];

- Where **type** can be any valid C data type and **arrayName** will be a valid C identifier.

Two-dimensional Arrays

- A two-dimensional array can be considered as a table which will have x number of rows and y number of columns. A two-dimensional array **a**, which contains three rows and four columns can be shown as follows –

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Thus, every element in the array **a** is identified by an element name of the form **a[i][j]**, where 'a' is the name of the array, and 'i' and 'j' are the subscripts that uniquely identify each element in 'a'.

Two-dimensional Arrays

For example : **float x[3][4];**

- Here, x is a two-dimensional (2d) array. The array can hold 12 elements. You can think the array as a table with 3 rows and each row has 4 columns.

	Column 1	Column 2	Column 3	Column 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]

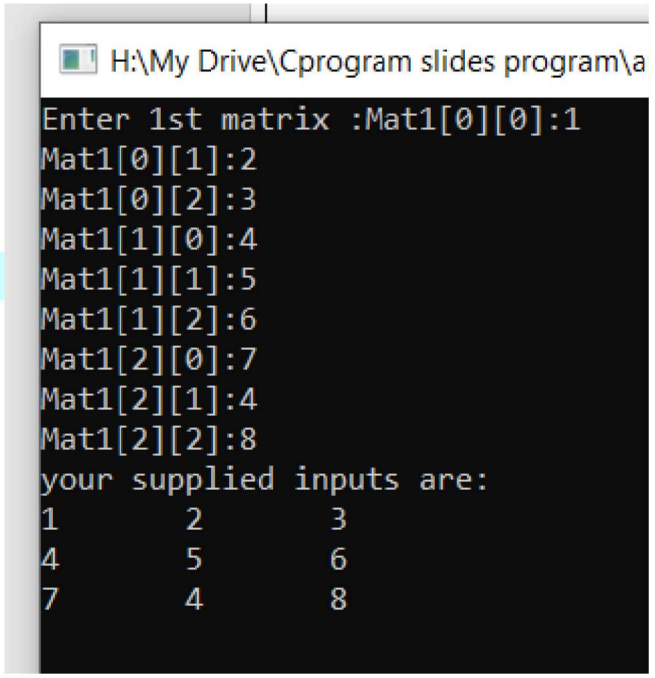
QN7: Program to demonstrate the loading or storing of data in two dimensional array.

```c
#include<stdio.h>
int main()
{
    int a[3][4],i,j;
    for(i=0;i<3;i++)
    {
        for(j=0;j<4;j++)
            a[i][j]=i+j;
    }
    for(i=0;i<3;i++)
    {
        for(j=0;j<4;j++)
            printf("a[%d][%d]=%d ",i,j,a[i][j]);
        printf("\n");
    }

    return 0;
}
```

```
H:\My Drive\Cprogram slides program\arrays\7.exe
a[0][0]=0 a[0][1]=1 a[0][2]=2 a[0][3]=3
a[1][0]=1 a[1][1]=2 a[1][2]=3 a[1][3]=4
a[2][0]=2 a[2][1]=3 a[2][2]=4 a[2][3]=5

--------------------------------
Process exited after 0.05597 seconds with return value 0
Press any key to continue . . .
```

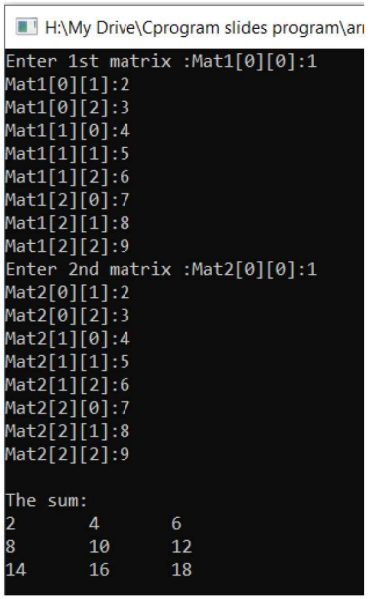
Program that reads the 3x3 matrix and display the elements in that matrix

```c
#include<stdio.h>
int main()
{
    int mat1[3][3],mat2[3][3],sum[3][3],i,j;
    printf("Enter 1st matrix :");
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("Mat1[%d][%d]:",i,j);
            scanf("%d",&mat1[i][j]);
        }
    }
    printf("your supplied inputs are:\n");
    for(i=0;i<3;i++)
        {
            for(j=0;j<3;j++)
               printf("%d\t",mat1[i][j]);
            printf("\n");
        }
}
```



QN8: Program to read two matrices and display their sum and differences.

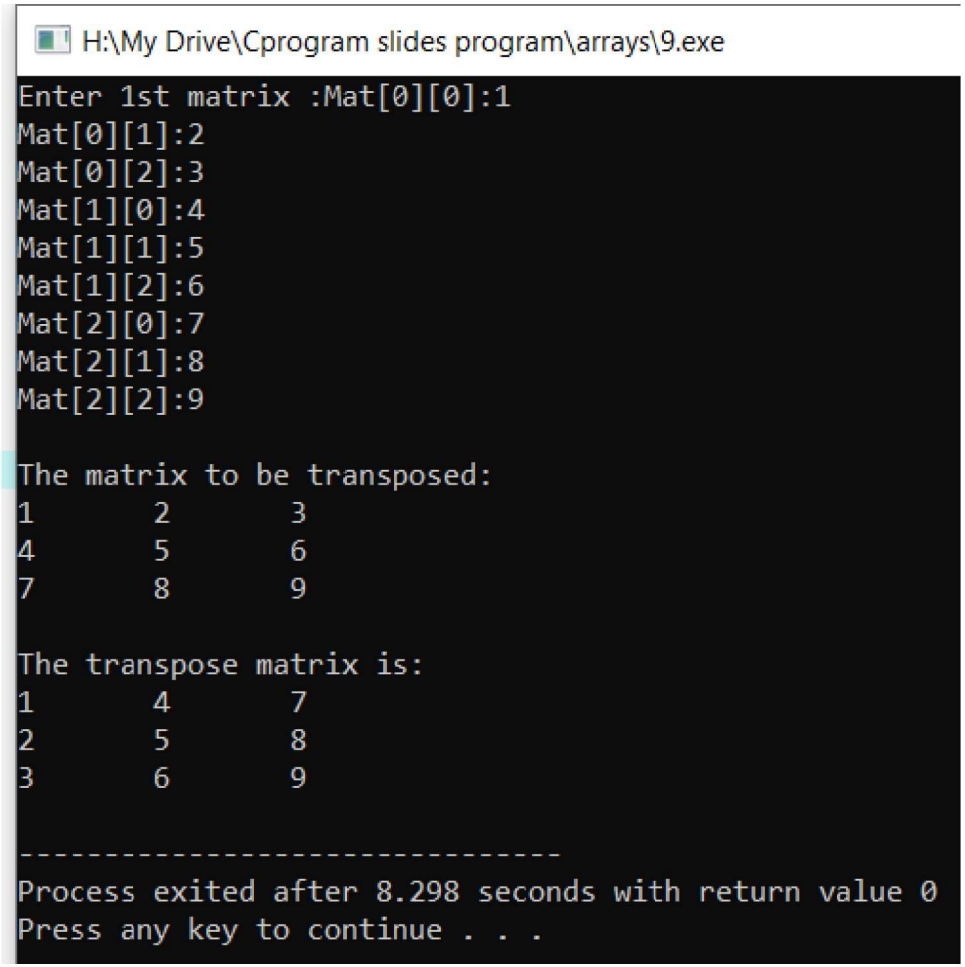
```
#include<stdio.h>
int main()
{
   int mat1[3][3],mat2[3][3],sum[3][3],i,j;
   printf("Enter 1st matrix :");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("Mat1[%d][%d]:",i,j);
         scanf("%d",&mat1[i][j]);
      }
   }
   printf("Enter 2nd matrix :");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("Mat2[%d][%d]:",i,j);
         scanf("%d",&mat2[i][j]);
      }
   }
   for(i=0;i<3;i++)
      {
         for(j=0;j<3;j++)
         {
            sum[i][j]=mat1[i][j]+mat2[i][j];
         }
      }
   printf("\nThe sum:\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
        printf("%d\t",sum[i][j]);
      printf("\n");
   }
   return 0;
}
```



QN9: Program to find transpose matrix.

```
#include<stdio.h>
int main()
{
   int mat[3][3],tran[3][3],sum[3][3],i,j;
   printf("Enter 1st matrix :");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("Mat[%d][%d]:",i,j);
         scanf("%d",&mat[i][j]);
      }
   }
   printf("\nThe matrix to be transposed:\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
        printf("%d\t",mat[i][j]);
      printf("\n");
   }
 for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         tran[j][i]=mat[i][j];
      }
   }
   printf("\nThe transpose matrix is:\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
        printf("%d\t",tran[i][j]);
      printf("\n");
   }
   return 0;
}
```

o/p


H:\My Drive\Cprogram slides program\arrays\9.exe
Enter 1st matrix :Mat[0][0]:1
Mat[0][1]:2
Mat[0][2]:3
Mat[1][0]:4
Mat[1][1]:5
Mat[1][2]:6
Mat[2][0]:7
Mat[2][1]:8
Mat[2][2]:9
The matrix to be transposed:
1 2 3
4 5 6
7 8 9
The transpose matrix is:
1 4 7
2 5 8
3 6 9

Process exited after 8.298 seconds with return value 0
Press any key to continue . . .

QN10: Program to find 3x3 matrix multiplication.

```
#include<stdio.h>
int main()
{
   int mat1[3][3],mat2[3][3],sum[3][3],i,j,k;
   printf("Enter matrix 1:\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("Mat1[%d][%d]=",i,j);
         scanf("%d",&mat1[i][j]);
      }
   }
   printf("Enter matrix 2:");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("Mat1[%d][%d]=",i,j);
         scanf("%d",&mat2[i][j]);
      }
   }
```

```
   printf("\nYour matrix 1 is :\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("%d\t",mat1[i][j]);
      }
      printf("\n");
   }
   printf("\nYour matrix 2 is :\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("%d\t",mat2[i][j]);
      }
      printf("\n");
   }
```

Contd....

```
//Multiplication calculation
  for(i=0;i<3;i++)
  {
     for(j=0;j<3;j++)
     {
        sum[i][j]=0;
        for(k=0;k<3;k++)
        {
           sum[i][j]=sum[i][j]+mat1[i][k]*mat2[k][j];
        }
     }
  }
  printf("\nMultiplication :\n");
  for(i=0;i<3;i++)
  {
     for(j=0;j<3;j++)
     {
        printf("%d\t",sum[i][j]);
     }
     printf("\n");
  }
  return 0;
}
```

QN10: Program to find 3x3 matrix multiplication.

```
#include<stdio.h>
int main()
{
   int mat1[3][3],mat2[3][3],sum[3][3],i,j,k;
   printf("Enter matrix 1:\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("Mat1[%d][%d]=",i,j);
         scanf("%d",&mat1[i][j]);
      }
   }
   printf("Enter matrix 2:");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("Mat1[%d][%d]=",i,j);
         scanf("%d",&mat2[i][j]);
      }
   }
```

```
   printf("\nYour matrix 1 is :\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("%d\t",mat1[i][j]);
      }
      printf("\n");
   }
   printf("\nYour matrix 2 is :\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("%d\t",mat2[i][j]);
      }
      printf("\n");
   }
```

```
   //Multiplication calculation
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         sum[i][j]=0;
         for(k=0;k<3;k++)
         {
            sum[i][j]=sum[i][j]+mat1[i][k]*mat2[k][j];
         }
      }
   }
   printf("\nMultiplication :\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("%d\t",sum[i][j]);
      }
      printf("\n");
   }
   return 0;
}
```

QN11: Program to find the sum of square in a diagonal of a square matrix matrix.

```
#include<stdio.h>
int main()
{
   int mat[3][3],sum=0,i,j;
   printf("Enter matrix :\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("Mat[%d][%d]=",i,j);
         scanf("%d",&mat[i][j]);
      }
   }
   printf("\nYour matrix  is :\n");
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         printf("%d\t",mat[i][j]);
      }
      printf("\n");
   }
```

```
 //calculation
   for(i=0;i<3;i++)
   {
      for(j=0;j<3;j++)
      {
         if(i==j)
            sum=sum+mat[i][j]*mat[i][j];
      }
   }
   printf("\nThe sum of square of
element on a diagonal is : %d",sum);
   return 0;
}
```

String

Finished

Unit 5